Git for Science

A substrate for reproducible, compounding scientific knowledge

G. Barbalinardo
University of California, Davis

June 7, 2026

Git did not just store files. It stored the *repository*: a structured, content-addressed object with identity and lineage, of which the working files are one view. That object is why merging, provenance, and collaboration became mechanical, and why a planet's worth of code could accumulate in one shared form. Science is still pre-git. We ship the paper, the printout of a result, and throw the object away. This is an attempt to store the object: what a result *is* (a typed physical quantity, its formula, its lineage, and the code that produced it as one representation of it), so that results become reproducible by construction, knowledge compounds instead of evaporating, and an AI agent has a semantic place to act.

The creed

1. **The unit of scientific knowledge is meaning, not text.**
2. **Physics is the invariant. A code is one representation of it.**
3. **A scientific result is a structured object with identity and lineage, not a number in a table.**
4. **A result you cannot re-run is a claim, not yet knowledge.**
5. **Units, gauge, and lineage are part of a result, not footnotes to it.**
6. **Knowledge should compound, not evaporate into scripts.**
7. **Correctness should be structural, checked by the substrate, not re-derived by trust.**
8. **Verification must scale with compute, not with reviewers.**
9. **The native substrate for scientific AI is semantic, not token text.**

The problem: science that does not accumulate

Computational materials science is a Tower of Babel. A single physical quantity, say the lattice thermal conductivity of a crystal, is computed by a dozen codes: kaldo, phono3py, ShengBTE, LAMMPS, VASP, Quantum ESPRESSO. Each has its own input formats, its own unit conventions, its own gauge and basis choices, and its own unspoken assumptions about what was held fixed. The physics is the same. The codes never quite agree, and deciding whether two of them *should* agree is itself expert work.

That work does not accumulate. Reconciling two calculations is artisanal, done by a handful of people who carry the conventions in their heads, and most published computational results are effectively irreproducible because the real workflow lives in tribal knowledge

and one-off scripts rather than in the paper. The paper is the tarball we email. Once it is sent, the object that produced it is gone.

The diagnosis: meaning is implicit

The reason the codes do not agree mechanically is that their meaning is implicit. Scientific knowledge today lives in three forms, and each loses something. Prose, in papers, carries rich meaning but does not run. Code runs, but its meaning is entangled with implementation: the physics is mixed in with array layouts, loop orders, and file formats. Data is numbers stripped of their provenance. None of the three carries meaning that is at once machine-checkable and runnable.

So the physics is never stored directly. It is buried inside whichever artifact happened to carry it, and recovering it, deciding what a given number actually claims, is left to the reader. What cannot be recovered mechanically cannot be checked mechanically, and so it cannot accumulate.

The bet: store the physics, not the printout

The move is to store the object. A result becomes a typed quantity, thermal conductivity or a phonon frequency or a heat capacity, declared as what it is. The formula that produces it rides on the edge that produces it, written symbolically rather than implied by code, and the quantity carries its lineage, the ordered chain of operations behind it, along with its units and gauge. A code is then no longer the result; it is one *representation* of it, a projection of a single shared operator layer into one discretization and one set of conventions.

Every representation factors through that shared layer, never code to code. That is what makes reconciliation mechanical: two codes that computed the same observable are two projections of one object, so comparing them is a structural operation on the graph rather than a feat of expert translation. It is merge, for science.

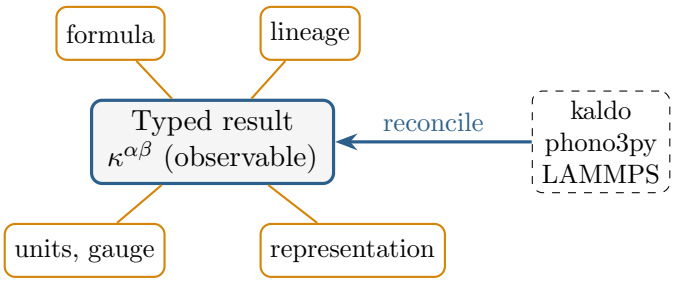

Figure 1: A scientific result as a structured object: a typed quantity carrying its formula, lineage, units and gauge, with each code a representation that reconciles into it at the observable. Merge, for science.

This is the same decision git made. A commit is identified by its content and its parents; a result here is identified by its type and its lineage. Git won on its object model, the blobs and trees and commits, not on its interface, and the contribution here is the analogous object model for a scientific result. Getting that object right is the whole game.

One difference is worth stating plainly, because it is where the real work lives. Code is discrete and born digital, while a scientific result carries approximation, continuous quantities, and gauge freedom: a per-mode lifetime depends on a basis choice no two codes share, even though the conductivity it sums to does not. That is why the object is typed, why units travel with it, and why the layer separates *observable* quantities, which must agree across codes, from representation-dependent scaffolding, which need not. Defining what is allowed

to differ is as much of the design as defining what must match. The project already ingests a *code* this way; the vision is to ingest a *paper* the same way, turning its claims into objects the substrate can hold, re-run, and check.

Proof it is real

None of this is a promise about future software; the substrate runs today. Take the lattice thermal conductivity of silicon. Computed through the operator graph, it comes out directly in watts per meter per kelvin, with the unit conversion applied automatically from the typed dimensions rather than pasted in by hand. The same conductivity assembled along two different routes, the total on one side and the sum of its population and coherence parts on the other, agrees to within about $10^{-6}\,\mathrm{W/(m\,K)}$ on a value of order 100, because the two routes are views of one object and the substrate holds them to it.

The repository ships a smaller version you can run in seconds. `examples/quickstart.py` builds the operator graph (46 quantities, 47 operators), derives the molar heat capacity of a crystal from a phonon frequency array, $11.83\,\mathrm{J/(K\,mol)}$ at $300\,\mathrm{K}$, and cross-checks two independent inputs at a gauge-invariant observable; the validation report returns expected agreement with zero residual. This is merge, for science, and it already runs.

The live operator graph is at the project page; the architecture is documented in `operator_representation_substrate.`

The payoff: knowledge that compounds

When results live as typed, reproducible objects, they stop being isolated. Each one added to the graph makes the whole denser: a new code is a new representation that can be checked against the others, and a new quantity opens new routes that cross-validate the old ones. Reproducibility is the proof that a result was captured faithfully; compounding is the dividend. The value is no longer locked in any single paper but accrues in the shared, checkable object that all of them project into.

This is the network effect git set off. Git made a standard object for code, and on top of it grew GitHub, continuous integration, pull requests, and code review, an entire ecosystem that exists because the object underneath was shared and mechanical. A standard object for science invites the same: reproducibility infrastructure, automated review, and a body of physics that grows more valuable, and more trustworthy, with every result poured into it.

The horizon: a semantic action space for AI

There is a reason to care about this beyond tidiness. Today's models reason about science in token space: they manipulate text that describes physics, and they re-derive correctness from statistical pattern every time they are asked. That is why a model can produce a fluent, confident, wrong step whose error only surfaces three papers downstream. The substrate underneath is lossy, so the reasoning on top cannot be trusted by construction.

A typed substrate offers a different footing: a semantic action space. An agent that acts on the graph does not write a description of a calculation; it composes an edge, materializes a quantity in a chosen code, contracts a hidden quantity into an observable, cross-checks two routes. Each move is type-checked and dimensionally sound by construction, so an invalid step becomes a type error rather than a silent number. And because the routes are redundant, the substrate is its own verifier, which is the scarce ingredient for reinforcement learning in any open-ended domain: an agent can search and be told, mechanically, whether the answer holds.

The analogy closes the loop. The structured corpus of code that accumulated in git is

what made today's code models possible; a structured corpus of science is what will let an agent reason over physics rather than over prose about physics. This is, deliberately, Lean for computational physics, and like a proof assistant its worth is that correctness is checked, not trusted. It is only partly built, but the shape is already here.

Where we are, and what is next

Today the framework is real and small. It has an operator layer and a representation layer, seven ingested codes (kaldo, phono3py, phonopy, ShengBTE, ASE, LAMMPS, and GPUMD), and a validation engine that executes the graph, composes edges symbolically, and cross-checks redundant routes. Units reconcile automatically from the typed dimensions. The worked domain is lattice thermal transport, a graph of 46 quantities and 47 operators, and the whole is held together by a suite of 609 passing tests.

What is next is mostly upward and outward. The upstream from the Born-Oppenheimer potential, the force constants from which everything descends, is currently stubbed rather than modeled in full. The symbolic composition of error, the approximation error carried by an operation and the discretization error carried by a representation, is designed and not yet built. And the largest reach, the one this document is really about, is to ingest not only codes but whole papers: to turn a published result into an object the substrate can hold, re-run, and check, the same way it already ingests a code.

Git gave code a standard object, and an ecosystem grew on it. Give science the same, and a result stops being a dead end: it becomes a contribution to a compounding, machine-usable, verifiable body of physics.

The standard to store science.